2023 IEEE 3rd International Conference on Electronic Technology, Communication and Information (ICETCI) | 979-8-3503-9841-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICETCI57876.2023.10176551

A Method for Robot Indoor Path Planning Using BIM Based on A* Algorithm

Yujie Hou*
The Hong Kong University of Science and Technology
Clear Water Bay, Kowloon, Hong Kong, China
yhouap@connect.ust.hk

Yuting Ju
North University of China
Taiyuan, China
xiaoju021@163.com

***Abstract*—As for map generation, the robot usually scans the surrounding building environment through its lidar camera and converts the 3D obstacle information into 2D grid information to build a map model. The advantage of this method is that it can significantly compress unnecessary information and improve the efficiency of information utilization. But at the same time, for some cases where it is not possible to go to the site for scanning, this method which requires field scanning, is not applicable. To improve this situation, it generally uses the Gazebo software in ROS (Robot Operating System) to reconstruct the building scenes that need to be converted to map information and simulate the process of lidar camera scanning to generate 2D grid maps. However, the problem with this method is that it is not possible to make the dimensions of the reconstructed buildings in Gazebo the same as the original building. To solve this problem, this project uses the existing BIM model of the building to import into Gazebo's reconstructed environment and adjusts the scale so that the reconstructed model simulated in the software matches the real size, thus achieving the maximum reproduction of the real environment. For robot navigation, the most important thing is the path planning of the robot. The A* algorithm is currently the most commonly used in implementing global path planning. In this project, the mathematical aspects of the heuristic function in the A* algorithm are optimized, and the effect of the optimization on the computational speed of path generation is compared before and after the optimization to verify the impact of the optimization and improve the computational speed.**

***Keywords*—Map generation, 3D obstacle information, Robot Operating System, robot navigation**

I. Introduction

Today's society is developing rapidly in science and technology, and more and more high-tech products are appearing in people's production life. Robots have been used more and more frequently in daily life in recent years. From home life to the military field, you can see various robots everywhere. The massive demand for robots has contributed significantly to robotics development, and a wide variety of robotics companies have sprung up. The different robotic products they produce have considerably improved people's productive lives [1].

In robot development, intelligence has gradually become an important indicator to measure robots, of which autonomous mobility is a typical feature of robot intelligence. The application field of autonomous mobile robots is quite extensive. The widespread use of intelligent autonomous mobile robots in civil, military, and scientific areas confirms the rapid development of artificial intelligence over the years. Autonomous mobile robots have played a massive role in detecting complex environments, transporting factory goods, and operations in high-risk environments [2].

Autonomous mobile robots are a combination of sensor detection technology, automation technology, data fusion [3] technology, traditional mechanical design and electronics, and other cutting-edge technologies, which can be said to be the crystallization of high human technology with functions such as environment perception [4], navigation planning and behavior control[5].

According to the literature, it is found that Simultaneous localization and mapping (SLAM) is currently an effective method to deal with autonomous localization and navigation of robots in unknown environments [6]. This thesis focuses on the path navigation systems in indoor scenes, using BIM models for environment building, and robots using external sensors to collect environment information, generate environment maps, locate their position and navigate walking, and improve the speed effectively of the algorithm [7].

This experiment is mainly aimed at the robot path planning of large public areas and places, and the main obstacles are structures such as walls and load-bearing columns, which are reflected in the BIM model. This study focused on an experimental site, the Academic Building of the Hong Kong University of Science and Technology [8].

The experimental site is mainly the Chia-Wei Woo Academic Concourse on the first floor of the Academic Building. The simulation area is about 600 meters * 135 meters of irregular area, the total area selected is about 830 square meters(Fig.1).

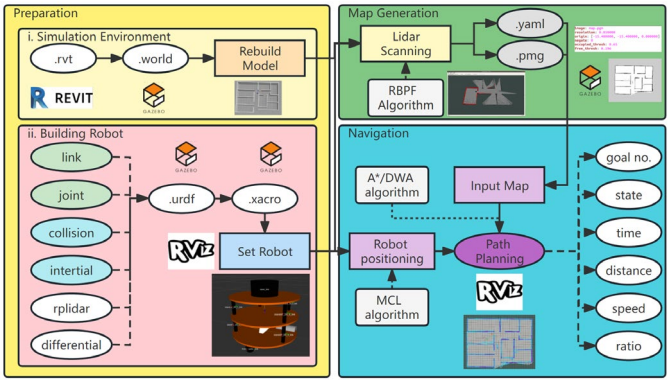


Fig.1. The workflow of BIM conversion applying to SLAM

The main reasons for choosing this place are as follows:

1) The location is wide enough for the robot to move and test inside.

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.

2) The main obstacles at this location are walls and load-bearing columns, where the edges of the obstacles are more irregular and more suitable for testing the accuracy of map generation and the functionality of navigation and obstacle avoidance [9].

3) The location is a public area of the Hong Kong University of Science and Technology. The simulated environment is more in line with the actual environment of the robot work; the simulation results have some reference value for the actual situation.

II. Preparation

Reconstructing the Model from BIM to Gazebo: Before performing a simulation, a simulation environment needs to be built. The leading software used for this process is Gazebo, where there are two ways to create a simulation environment [10]. One is to insert models directly from the model list in Gazebo, including some robots and external objects cases, and the other is to draw maps manually from the Building Editor in Gazebo [11].

Since one of the purposes of this project is to combine the BIM model environment information with Rviz's navigation simulation [12], we choose the 2nd way in Fig.1. In Gazebo, select Edit - Building Editor to open the drawing interface.

In this project, the information about the BIM model of the simulated site is imported into the upper window, and the appropriate size conversion rate is selected to match the model size in Gazebo with the real size in the BIM model. It can be seen that the imported BIM model dimensions correspond to the scale in the bottom right corner of Gazebo in Fig.2.



Fig.2. After importing BIM information into the Gazebo drawing area



Fig.3. The drawing window in Gazebo

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.

Next, start the drawing of the simulation environment. Select the wall mode on the right side of Building Editor and start drawing along the boundary of the BIM model in the upper side window. The thickness of the wall can be set by right-clicking and selecting Properties. For rounded columns and irregular edges, you can insert the cylindrical obstacle that comes with Gazebo to the specified position and set the appropriate size. The window on the bottom side allows you to monitor the drawing situation. The upper side window draws along the BIM model, and the lower side window monitors the drawing, as shown in Fig.3.

The initial conversion of the map information from BIM to Gazebo is completed when the model is built. The finished model is displayed in Gazebo, as shown in Fig. 4. The next step is to load the robot model and perform the simulation.

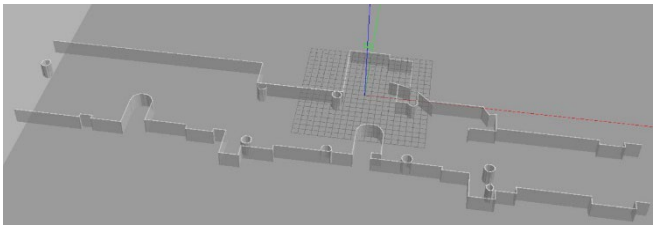


Fig.4. Site model after conversion in Gazebo

III. 2D Map Generation

A. Gmapping Packege in SLAM

The map generation process mainly relies on SLAM (Simultaneous Localization and Mapping) technology. SLAM can be described as a robot moving from an unknown position in an unknown environment. During the movement, it performs its localization based on position estimation and maps while building incremental maps to achieve autonomous robot localization and navigation. Autonomous navigation is often inseparable from SLAM because the map generated by SLAM is the main blueprint for independent robot movement [13]. Therefore, this phase of map generation is significant for the next navigation phase. To complete the progress of SLAM and navigation, the robot must first have the ability to perceive the surrounding environment, especially the ability to perceive the depth information of the surrounding environment, because this is the key data for obstacle detection. For this project, we used LIDAR. LIDAR is the most researched and used depth sensor, providing information about the distance between the robot body and the environmental obstacles. It has the advantages of high accuracy, fast response, and small data.

The ROS open source community has a variety of SLAM algorithms that can be used directly or for secondary development, the more common and mature of which is the Gmapping package, which integrates the Rao-Blackwell list particle filtering algorithm, eliminating the need for a complex internal implementation for developers. The package transforms the depth, IMU, and mileage information collected by the robot's top camera to create and output a probability-based 2D raster map. The 2D raster map is the map format that the robot can directly utilize during navigation. This project uses the Gmapping function package to complete the conversion of map information.

B. 2D Map Generation

Since we have used the lidar camera packet that comes with ROS in the previous stage, we only need to manipulate the robot's actions to complete the map reconstruction in this stage. First, we import the reconstructed wall obstacle model from Chapter 2 into Gazebo. Next, we need to put the robot model equipped with the depth sensor made in the previous section into the simulation environment. And use the lidar camera as the depth sensor to start the simulation environment.

After the start of the simulation environment, we activate the Gmapping navigation package. The robot model in Gazebo can be seen stationary in the middle of the simulation environment, and the scan results are displayed simultaneously in Rviz. The red dot in front of the robot is the depth information of the 2D environment detected by the lidar sensor in real-time. The machine creates a map of part of the known environment in light gray based on the current depth information in Fig. 5.

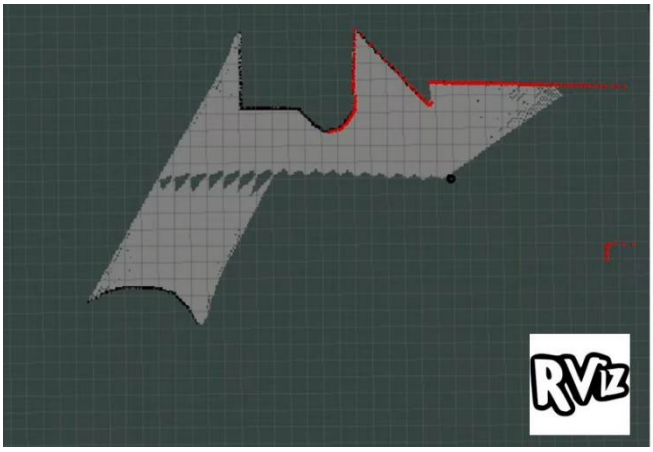


Fig.5. Scanning and using the depth information to generate a 2D map

The keyboard is then activated to control the robot's movement through the map. As the robot moves, the map in Rviz is continuously updated and Gmapping automatically corrects for deviations between the previously created map and the robot's position in Fig. 6.

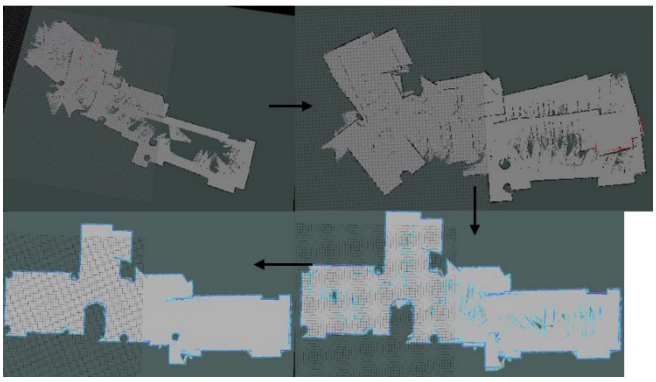


Fig.6. Gradual improvement in map quality with continuous correction

The map is basically built after controlling the robot around the environment for a week. Next, save the map. The map is saved to the directory of the current terminal, named map by default, and contains the map data file and other information files. The map.pgm file is the map data file. map.yaml is the configuration file for the map, which contains the resolution, starting position, and other configuration parameters. Using these two files generated at this stage, we can carry out the next navigation step.

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.

IV. NAVIGATION

The key to robot navigation is the two main parts of robot localization and path planning. This chapter will explore these two problems, focusing on improving the A* algorithm used in robot path planning to enhance the computational speed of path generation.

A. Robot localization

Robot positioning is mainly how a robot derives its position after processing the information collected by lidar cameras.In the ROS system. The amcl function package is provided for robot localization. Firstly, the function package needs to contain the sensor information of the robot and convert this information into the pose and position information of the robot through specific calculations.

B. Robot path planning

1) Global path planning and local path planning

The path planning of robots is mainly divided into two categories: global path planning and local real-time planning.

Global path planning is the robot's overall path planning based on a given target location and a global map. This project uses the A* algorithm for global path planning to calculate the optimal route from the robot to the target location as a global route.

In practice, the robot often cannot follow the global route strictly, so it is necessary to plan the route that the robot should travel in each cycle for the map information and the obstacles that may appear near the robot at any time so that it conforms to the global optimal path as much as possible. This behavior of adjusting the path in real-time based on the information is called local path planning.

The A* algorithm for global path planning will be optimized in this project.

2) Improving the A algorithm*

A* is a heuristic algorithm, widely used in map traversal and pathfinding and is used in many large games. A* takes advantage of all known information in the path planning process, such as the number of iterations and the estimated cost from the starting location and the current location to the target location to select the following path. In path planning based on the occupied raster map, A* takes into account the currently known global information when selecting the raster cell to be examined next and evaluates the cost distance from the current raster cell to the target raster cell and the cost distance from the starting raster cell to the current raster cell as a criterion for evaluating the current raster cell on the optimal path. The basic idea of A* path planning: define the valuation function F(x) of the current grid cell x. The valuation function F(x) is defined as follows (1):

$$F(x) = G(x) + H(x) = G(x) + w \times Distance \qquad (1)$$

In the above equation, $G(x)$ represents the actual cost distance from the starting raster cell o to the current raster cell x, and $H(x)$ represents the minimum estimated cost distance from the current raster cell x to the destination raster cell d. The specific form of $H(x)$ can be defined according to the actual situation. The A* algorithm starts from the starting grid cell and searches for the grid cell with the smallest $F(x)$ value until the destination grid cell is searched.

The most important thing in the implementation of A* is the definition of the estimation function $H(x)$.

The mainstream method of defining $H(x)$ uses the Euclidean distance assuming the two-dimensional coordinates of the starting raster cell $o(o_x, o_y)$, the two-dimensional coordinates of the destination raster cell $d(d_x, d_y)$, and the two-dimensional coordinates of the current raster cell $x(x_x, x_y)$, then the estimation function $H(x)$ is defined as follows (2):

$$Distance = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \qquad (2)$$

Another way to calculate the distance is the diagonal distance method. The specific formula is as follows (3):

$$Distance = \sqrt{2} \times min(|x_1 - x_2|, |y_1 - y_2|) + max(|x_1 - x_2|, |y_1 - y_2|) - min(|x_1 - x_2|, |y_1 - y_2|) \qquad (3)$$

Compared with the Euclidean distance formula, the diagonal distance formula will have a more optimized route in the path generation process when applied to 2-dimensional raster maps. The path from the starting raster to the ending raster will have a more optimized route generated most shortly. Theoretically, using the diagonal formula to the definition of heuristic function in the A* algorithm will result in a more optimal algorithm.

In comparison, $H(x)$ is more biased to determine the computational speed of global path generation. The higher the weight of $H(x)$ in the navigation algorithm, the faster the navigation generates paths, the lower the corresponding navigation accuracy. The common weight value used in previous projects is 1.0.

This project will use the Euclidean distance method and the diagonal distance method respectively and increase the $H(x)$ weight to 3.5. The navigation effect will be compared with the navigation effect before changing the weight, including the accuracy of path generation and the computation speed in Fig.7.

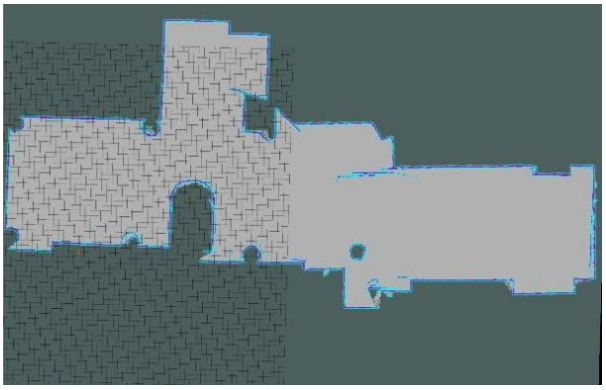


Fig.7. Loading Campus 2D Map

C. Deployment Navigation

In deploying the navigation, we need to start all the profiles in Gazebo, including the profile of the map and the map server, and load the set map. As shown in the picture, the purple part is where the wall is located. The blue area is the

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.

safety distance set to prevent the robot from collision, which is half of the body length in Fig. 8.

Also, we need to load the robot model created in section 2 in the navigation environment. Next, watch the start-up effect through Rviz. As you can see, the map file and the robot model are shown below after the data has been loaded. You can see that the red dots are where the robot is located, and the green arrows are its walking direction. The area marked in red is the location of the obstacle that the robot's lidar camera scanned in real-time.



Fig.8. Loading the robot model into the 2D map

Click the 2D Nav GO button in the menu bar with the mouse. The function of this button is to help set the navigation target point. Move the mouse to the target location for navigation on the map and click the left mouse button. Then it can be seen a green arrow at the target location. Because the navigation target point contains the robot's position information and the robot's pose information, it can set the pose of the navigation target by dragging the mouse. After determining the target and releasing the left mouse button, it can immediately be seen that the move_base function package has created an optimal path using the global planner between the robot's current position and the target position. Although the robot moves quickly, the robot cannot follow the optimal path exactly due to the limitations of the robot's physical parameters. There is a short red line near the robot, which is the optimal speed for the current cycle planned by the local planner for the robot, to try to ensure that the robot. Close to the globally optimal path, the robot will rotate to the specified attitude after reaching the target position, and the navigation process ends.

When opening the visual terminal for log monitoring, it is possible to see the logs of distance information, status information, target point number, success rate, and speed issued by the robot operation. In this project, a time limit value of 60 seconds is set. If the planning result is not obtained from the server within this limited time, the robot motion planning is not completed, and the planning is considered to have failed.

D. Comparison of navigation results before and after optimization

To compare the effect of two different distance calculation formulas and changes in weights on the computational rate of robot path planning in the same case, the same starting point and different ending points were selected on the map to control other variables of path planning in this project in Fig.9. The computed path length and computation time during this process are also recorded.

First, we set up three cases:

Case 1: the weights are not changed (i.e., w=1.0) and use the Euclidean distance estimation method;

Case 2: the weights are increased (i.e., w=3.0) and use the Euclidean distance estimation method;

Case 3: the weights are increased increase (i.e., w=3.0) and use the diagonal distance estimation method.

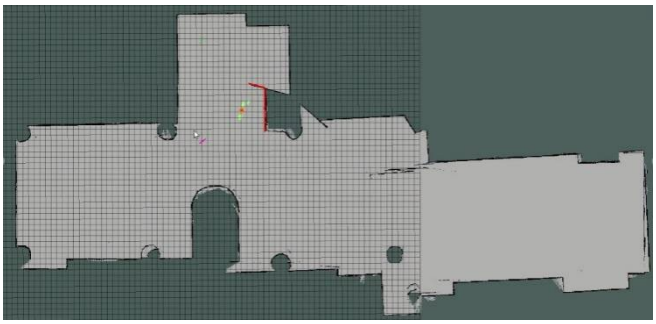


Fig.9. The same test points set in the three cases of this experiment

Simulation of the same driving path is performed for each of the three cases. The distance and calculation time of the path planning displayed in the corresponding terminal are recorded.

The computation times for each iteration for the three scenarios are obtained as follows in Table I:

TABLE I. COMPUTATIONAL TIME OF PATH GENERATION AT DIFFERENT DISTANCES AND CASES

No.	Case 1		Case 2		Case 3	
	Time (μSEC)	Distance(m)	Time (μSEC)	Distance(m)	Time (μSEC)	Distance(m)
1	33.70	3.15	7.34	3.16	3.20	3.17
2	30.43	3.12	6.47	3.12	2.55	3.12
3	25.87	3.03	4.75	3.01	2.10	3.00
4	18.27	2.86	3.64	2.86	1.80	2.81
5	11.04	2.69	2.69	2.69	1.60	2.70
Average	23.862	2.97	4.978	2.968	2.25	2.96

From the calculation time of case1 and case2, we can see that after the weight increased from 1.0 to 3.0, the calculation time dropped significantly, by about 79.1%; from the calculation time of case2 and case1, we can see that after the weight remained unchanged and the diagonal formula replaced Euclid's formula, the calculation time dropped slightly, by about 54.8%. Of course, in this process, due to the error of computer calculation, different generation paths will lead to a particular error in the calculated distance of the three cases. We can not guarantee that the calculated distance is the

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.

same time, but the difference value will not be significant, so the data still has a comparative value.

Based on the above data, a distance-time-linear plot was generated by fitting this project as follows Fig.10.

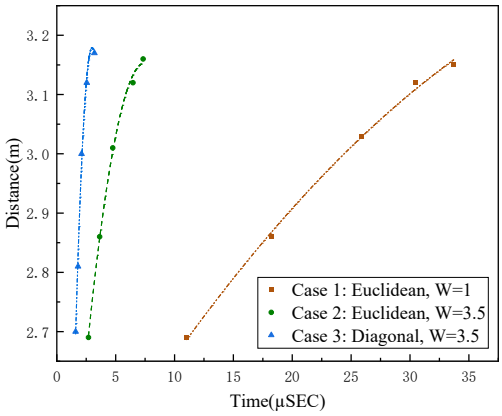


Fig.10. Computational time vs. distance fit for different cases

It can be seen that the slope of the fitted curve represents the calculation speed, in which the slope of Case 2 is significantly larger than that of Case 1, i.e., the calculation speed of Case 2 is considerably larger than that of Case 1. And the computational speed improvement is about 50.1%, according to the slope calculation, which indicates that increasing the weight significantly improves the calculation speed of path planning.

The slope of Case 3 is slightly larger than that of Case 2, i.e., the calculation speed of Case 3 is slightly larger than that of Case 2. And the computational speed improvement is about 16% according to the slope calculation, which indicates that using the diagonal distance formula instead of the Euclidean distance formula has a certain gain on the calculation speed of path planning.

V. Conclusions

With the rapid development of intelligent robots in recent years, this study takes the path navigation system of autonomous mobile robots as the research object, focusing on the map generation technology and path planning technology of autonomous mobile robots, and completes the study of robot path navigation in indoor environments based on the open source robot operating system ROS. The main research conducted includes the integration of BIM information with Gazebo simulation environment and the optimization of global path planning technology based on A* algorithm. The specific findings of this thesis are as follows:

This project made full use of the BIM model of HKUST and imported it into Gazebo software of the ROS system to complete the reconstruction of the model and map. The reconstruction is effective, with little difference in size from the original building, and the generated 2D raster map can be used for robot navigation, allowing the robot to simulate the navigation path based on existing information without field measurement, saving time and cost.

Also, this project optimizes the heuristic function for the A* algorithm of path planning. Replacing the Euclidean formula with the diagonal formula can improve the computational speed of path planning by about 16% while increasing the weight of the heuristic function from 1.0 to 3.0 can improve the computational speed of path planning by 50%. The overall optimized A* algorithm improves the computational speed by 67% compared with the original algorithm, and the optimization effect is remarkable.

References

[1] Yu, Deqin. (2017). "Domestic and International Industrial Robot Development Status and Trends." Polular Utilization of Electricity. (9), 2.

[2] Jin, Yaoqin, Jiang, Yongquan, and Tan, Bingyuan. (2017). "Intelligent robot status and development trend. " Computer & Telecommunication, (5), 27-28

[3] Zhang Hao, and Zhang Jingjing. (2017). "A Review of Data Fusion Algorithms for Wireless Sensor Networks." Computer Engineering & Software, 38(12), 9.

[4] Zhu, Bo, Gao, Xiang, and Zhao, Yannan. (2017). "A Review of Place-aware Approaches in Semantic Mapping of Robotic Interiors. " ACTA Automatica Sinica, 43(4), 16.

[5] Xu, Jingshuo, Zhou, Shengming, and Jiang, Huajun. (2009). "An Overview of The Development of Inertial Navigation System and Its Key Technologies. "Science & Technology Information, 000(035), 875-876.

[6] Guan, Xujun, and Wang, Xinlong. (2014). "Overview of the Development of Visual Navigation Technology. " Aero Weaponry (5), 7.

[7] Shi, Fengdong, Liu, Wenhao, Wang, Xin, Ding, Juan, Shi, Yijun, and Xiu, Chunbo. (2015). "Indoor Lidar Navigation System Design. " Infrared and Lidar Engineering, 44(12), 6.

[8] Bui HoangGiang et al. A variationally consistent hyperstatic reaction method for tunnel lining design[J]. International Journal for Numerical and Analytical Methods in Geomechanics, 2021, 46(1): 205-217.

[9] Mur-Artal, R., Montiel, J. M. M., & Tardos, J. D. (2015). ORB-SLAM: a versatile and accurate monocular SLAM system. IEEE transactions on robotics, 31(5), 1147-1163.

[10] Koenig, N., & Howard, A. (2004, September). Design and use paradigms for gazebo, an open-source multi-robot simulator. In 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566) (Vol. 3, pp. 2149-2154). IEEE.

[11] Meyer, J., Sendobry, A., Kohlbrecher, S., Klingauf, U., & Stryk, O. V. (2012, November). Comprehensive simulation of quadrotor uavs using ros and gazebo. In International conference on simulation, modeling, and programming for autonomous robots (pp. 400-411). Springer, Berlin, Heidelberg.

[12] Kam, H. R., Lee, S. H., Park, T., & Kim, C. H. (2015). Rviz: a toolkit for real domain data visualization. Telecommunication Systems, 60(2), 337-345.

[13] Pritsker, A. A. B. (1984). Introduction to Simulation and SLAM II. Halsted Press.

Authorized licensed use limited to: Karunya Institute of Technology and Sciences. Downloaded on July 02,2025 at 10:07:34 UTC from IEEE Xplore. Restrictions apply.